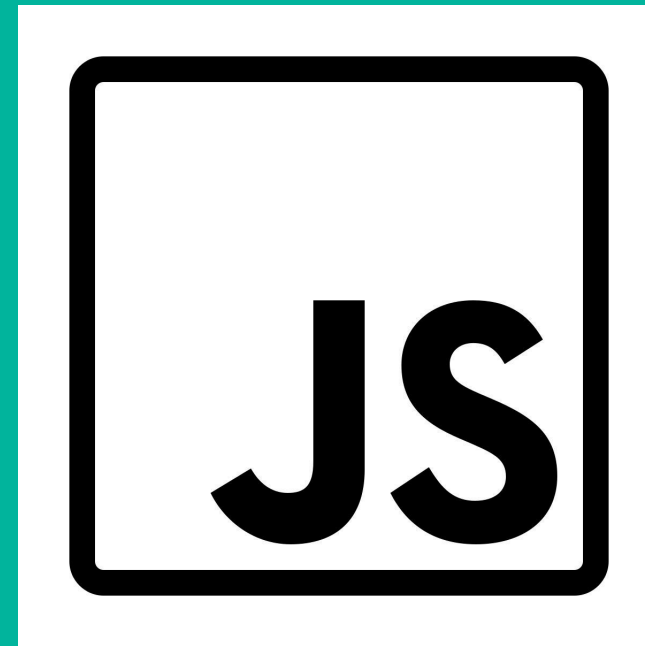




**VARIABLES, EXPRESSIONS,
OPERATORS, CONDITIONS,
OBJECTS, INHERITANCE**

JAVASCRIPT

Running code

- * **DevTools console**
- * **Javascript sandbox**
- * **Vscode + browser**
- * **<https://ej.uz/esd3> (W3Schools try it editor)**

Variables

- * **Named Memory Locations:** A variable acts as a labeled container pointing to a specific address in computer RAM.
- * **Identifiers & Rules:** Names must be descriptive (e.g., `userAge`), unique, and follow language rules (no spaces or starting numbers).
- * **Declaration vs. Assignment:** Creating the variable identifier vs. populating it with an initial value.
- * **Mutability:** Variable values can change over execution unless declared as immutable constants.
- * **Keywords** - `let`, `var`, `const`

Variable scope

* **Scope determines where variables are accessible in your code**

Scope Types

- **Global Scope:** Variables declared outside any function or block are accessible anywhere in the program.
- **Function Scope:** Variables declared with **var** inside a function are accessible only within that function.
- **Block Scope:** Variables declared with **let** or **const** inside curly braces `{}` (such as if statements or loops) are restricted to that specific block.

Variable hoisting

- * **Hoisting** is JavaScript's compilation mechanism where variable and function declarations are allocated memory at the top of their scope before execution. Only declarations are hoisted, not initializations.
- * **Function Declarations:** Fully hoisted along with their definition. You can invoke a function before declaring it in the source file.
- * **Variables:** Hoisted and initialized to *undefined*. Accessing them before initialization returns *undefined* instead of throwing an error.
- * **let and const Variables:** Hoisted to the top of their block, but left uninitialized in the Temporal Dead Zone.
- * **Temporal Dead Zone (TDZ)** is the execution timeframe between entering a scope and reaching a let or const variable's actual declaration line. Accessing the variable while it is in the TDZ throws a ReferenceError.

Variable hoisting

```
// Hoisting & TDZ Behavior Example

console.log(varVariable); // Output: undefined (hoisted & initialized)
var varVariable = "I am a var";

console.log(letVariable); // Throws ReferenceError (in TDZ)
let letVariable = "I am a let";

// Function Hoisting
greet(); // Output: "Hello!" (Fully hoisted)

function greet() {
  console.log("Hello!");
}
```

Variable storage

- * **Hoisting** is JavaScript's compilation mechanism where variable and function declarations are allocated memory at the top of their scope before execution. Only declarations are hoisted, not initializations.
- * **Function Declarations:** Fully hoisted along with their definition. You can invoke a function before declaring it in the source file.
- * **Variables:** Hoisted and initialized to *undefined*. Accessing them before initialization returns *undefined* instead of throwing an error.
- * **let and const Variables:** Hoisted to the top of their block, but left uninitialized in the Temporal Dead Zone.
- * **Temporal Dead Zone (TDZ)** is the execution timeframe between entering a scope and reaching a let or const variable's actual declaration line. Accessing the variable while it is in the TDZ throws a ReferenceError.

Data types

* Primitive data types

Fundamental built-in values, stored directly in execution memory for high speed

- * **Numbers** - JavaScript has no separate float, double or int types. All numbers are stored as 64-bit Double-Precision Floating-Point numbers
- * **Bigint** - A number representing a large integer
- * **String** - A text of characters enclosed in quotes
- * **Boolean** - true/false
- * **Null** - A value representing object absence
- * **Undefined** - A variable with no assigned value
- * **Symbol** - A unique primitive identifier

* Object types

- * **Object** - A collection of key-value pairs of data
- * **Array** - Array of values accessed by a numerical index
- * **Function** - reusable code blocks designed for particular tasks
- * **Date** - Date Objects let us work with dates
- * **RegExp** - sequence of characters that forms a search pattern
- * **Set** - collection of unique values
- * **Map** - collections of key-value pairs, similar to a dictionary
- * **Symbols** - represents a unique identifier.

Demo

<https://ej.uz/esd3>

Expressions

- * **Any valid phrase of literals, variables, functions and operators that evaluate to a single result value (e.g. $(x*4) + 10$)**
- * **Operator precedence:** Strictly dictates evaluation order (e.g. multiplication/division precedes addition/subtraction) Recommendation to use parentheses for clarity
- * **Side effects:** Expressions that modify state while returning a value (e.g. increment operators or function invocations)
- * **Type coercion:** Implicit conversion automatically conducted by runtime environment evaluating mismatched types (e.g. `int+float`)

Demo

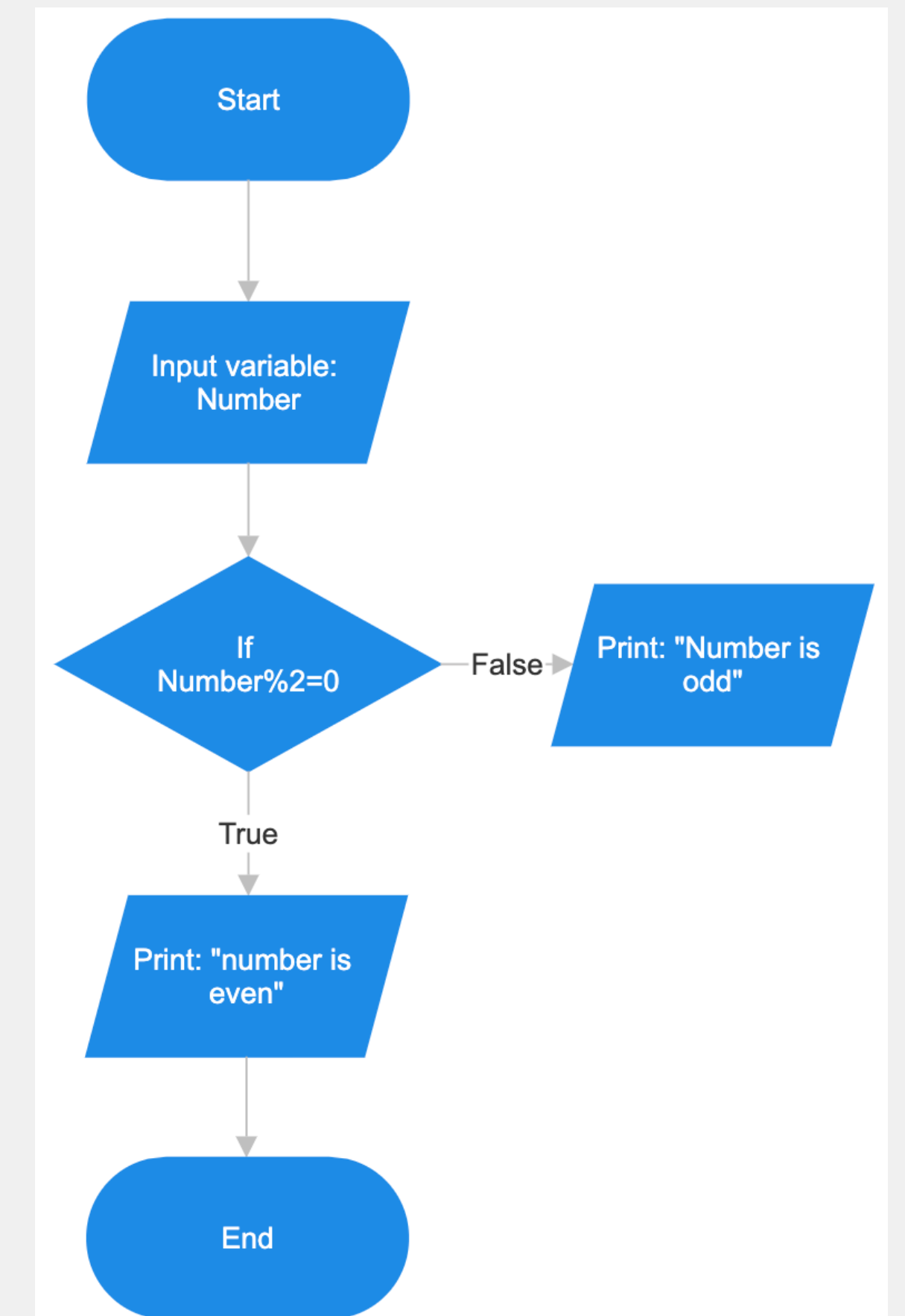
Operators

Category	Symbols	Purpose & Functionality	Example
Arithmetic	<code>+ - * / %</code>	Performs fundamental mathematical calculations	<code>total = price * 1.05;</code>
Relational	<code>== != > < >= <=</code>	Compares two operands; evaluates to a boolean result	<code>isAdult = (age >= 18);</code>
Logical	<code>&&</code> (AND) <code> </code> (OR) <code>!</code> (NOT)	Combines or inverts boolean logic conditions	<code>canAccess = (isMember && !isBanned);</code>
Assignment	<code>= += -= *=</code>	Evaluates right-hand side and assigns value to left target	<code>score += 10;</code>

Demo

Conditionals

- * Standard source code executes top-to-bottom sequentially. Conditional statements create decision forks, allowing dynamic paths.
- * By testing boolean expressions at runtime, programs react intelligently to user input, system states, and external data changes.



Conditionals

- * **If / else branching** - evaluates a primary condition, if true executes the primary block, otherwise falls back to else block
- * **Multi-branch (Else if)** - evaluates sequential rules top-to-bottom, execution enters only the first matching true block and skips all remaining branches
- * **Short-circuit evaluation** - logical AND (&&) stops immediately if the first term is false, logical OR (||) stops if the first term is true (boosting performance)

Demo

Objects & Classes

- * **Class (The blueprint)** - a user-defined template describing data structures and member functions without allocating object memory
- * **Object (The instance)** - concrete entity instantiated in memory and build according to the class specification
- * **State (Attributes)** - member variables stored inside an object representing the current values (e.g. color, speed)
- * **Behavior (Methods)** - functions belonging to an object that perform actions or alter internal state

Core OOP principles

- * **Encapsulation** - bundling state data and methods together while restricting direct external access to internal fields using access modifiers (private/public)
- * **Abstraction** - Hiding complex implementation details behind clean, simple interfaces presenting only essential interactions to external callers
- * **Instantiation** - allocating dynamic memory using construct calls (e.g. `new Student()`) to create distinct object identities

Demo

Inheritance

- * **Parent & Child Relationship:** Subclasses (children) inherit all attributes and methods from base classes (parents).
- * **Code Reusability (DRY):** Eliminates redundant code by centralizing shared behavior in high-level base classes.
- * **Method Overriding:** Child classes can provide specialized implementations of inherited parent methods.
- * **Polymorphism Introduction:** Treating different child objects uniformly as instances of their parent class type.

Inheritance examples

- * **Basic Class Inheritance (extends and super)** Modern JavaScript (ES6+) uses the class, extends, and super keywords to make inheritance simple.

```
// Parent Class (Base / Superclass)
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }

  // Parent method
  makeSound() {
    console.log(`${this.name} says "${this.sound}"`);
  }

  sleep() {
    console.log(`${this.name} is sleeping... Zzz`);
  }
}

// Child Class (Subclass) inheriting from Animal
class Dog extends Animal {
  constructor(name, breed) {
    // Call the parent class constructor to initialize 'name' and 'sound'
    super(name, "Woof!");
    this.breed = breed; // Property unique to Dog
  }

  // Method unique to Dog
  fetch() {
    console.log(`${this.name} the ${this.breed} fetches the ball!`);
  }
}

// --- Usage ---
const myDog = new Dog("Buddy", "Golden Retriever");

myDog.makeSound(); // Output: Buddy says "Woof!" (Inherited from Animal)
myDog.sleep();     // Output: Buddy is sleeping... Zzz (Inherited from Animal)
myDog.fetch();     // Output: Buddy the Golden Retriever fetches the ball! (Dog method)
```